

Lab 08: Deep Temporal Planning and Gridworlds

Objective

Implement multi-step policy evaluation, run MMP for deep inference, and build a gridworld agent with long-horizon planning.

Prerequisites

- Completed Labs 01–07
- Understanding of EFE accumulation and multi-step policies

Part 1: Multi-Step Policy Evaluation

Goal: Evaluate multi-step policies by unrolling predicted state trajectories.

1. Create a 2-state model with identity A, swap-or-stay B, $C = [2, -2]$, uniform D.
2. Define policies: `[[0, 0], [0, 1], [1, 0], [1, 1]]` (all 2-step combinations).
3. For each policy, compute total EFE by accumulating G_t across steps.
4. Print a ranked table of policies by total G.

```

from active_inference.math import compute_efc

policies = [[0, 0], [0, 1], [1, 0], [1, 1]]
for pi in policies:
    total_G = 0
    q = model.D.copy()
    for action in pi:
        G_t = compute_efc(q, model.A, model.B, model.C, action)
        total_G += G_t
        q = model.B[:, :, action] @ q
    print(f"Policy {pi}: G = {total_G:.4f}")

```

Response:

Write your response here

Part 2: Marginal Message Passing

Goal: Run MMP on a 3-step observation sequence.

1. Using the model from Part 1, create an observation sequence [0, 1, 0].
2. Run `run_mmp()` with a 3-step policy.
3. Print the beliefs at each time point.
4. Visualize convergence using `plot_convergence()`.

```
from active_inference.math import run_mmp
from active_inference.visualization import plot_convergence

result = run_mmp(
    prior=model.D, observations=[0, 1, 0],
    A=model.A, B=model.B, policy=[0, 1, 0],
)

for t, beliefs in enumerate(result["beliefs"]):
    print(f"t={t}: q(s) = {beliefs}")

plot_convergence(result["delta_history"],
                 save_path="output/lab08_mmp_convergence.png")
```

Response:

Write your response here

Part 3: Gridworld Environment

Goal: Build a 4×4 gridworld and visualize it.

1. Create a 16-state environment (flattened 4×4 grid).
2. Define 4 actions (up, down, left, right) with walls at positions (1,1) and (1,2).
3. Set C to prefer state 15 (bottom-right corner).
4. Visualize with `plot_gridworld()`.

Response:

Write your response here

Part 4: Planning Agent on the Gridworld

Goal: Run an agent with multi-step policies on the gridworld.

1. Define 3-step policies (a subset of all $4^3 = 64$ possible).
2. Create an `ActiveInferenceAgent` with these policies and $\gamma = 4.0$.
3. Run the agent for 20 steps.
4. Visualize the trajectory using `plot_gridworld()` with the path overlay.

Response:

Write your response here

Part 5: T-Maze with Temporal Depth

Goal: Compare T-maze performance with $T = 1$ vs $T = 2$ policies.

1. Set up the T-maze model from Module 05.
2. Run with single-step policies ($T = 1$): how often does the agent reach the reward?
3. Run with two-step policies ($T = 2$): e.g., [go-cue, go-left], [go-cue, go-right], etc.
4. Compare success rates across 20 trials.

Response:

Write your response here

Summary

Skill	Library Component	Status
Evaluate multi-step policies	<code>compute_eef()</code> in loop, policy unrolling	
Run deep temporal inference	<code>run_mmp()</code>	
Build gridworld environments	<code>DiscreteEnvironment</code> with grid B-matrix	
Run planning agents on gridworlds	<code>ActiveInferenceAgent(policies=...)</code>	
Compare planning depths	T=1 vs T=2 policy performance	